Distributed Systems Lab: Practical Exam Kit (ITE10301B)

Sem V, TY B.Tech IT (2024 Pattern). All programs in Java. Built around your 10 lab assignments.

How to use: (A) learn the glossary, (B) go through the likely questions, (C) memorise the short programs with their sample output. Every program is kept as small as possible so you can write it from memory. Parts 2 and 3 (programs 4 to 10) follow below.

A. Full Forms and Key Terms (practical concepts only)

Full forms

Short	Full form
RMI	Remote Method Invocation
RPC	Remote Procedure Call
CORBA	Common Object Request Broker Architecture
ORB	Object Request Broker
IDL	Interface Definition Language
IIOP	Internet Inter-ORB Protocol
GIOP	General Inter-ORB Protocol
POA	Portable Object Adapter
JRMP	Java Remote Method Protocol
MOM	Message-Oriented Middleware
DCE	Distributed Computing Environment
IPC	Inter Process Communication
MPI	Message Passing Interface
NTP	Network Time Protocol

Short	Full form
TTL	Time To Live
LRU	Least Recently Used
LB	Load Balancer
HTTP	HyperText Transfer Protocol
NFS	Network File System
HDFS	Hadoop Distributed File System
QoS	Quality of Service
CDN	Content Delivery Network
JVM / JDK	Java Virtual Machine / Java Development Kit

Definitions

- **Distributed system:** a collection of independent computers that appears to its users as a single coherent system.
- **Middleware:** software layer between the operating system and applications that hides heterogeneity and gives a uniform communication model (RMI, CORBA, MOM).
- **Content-centric middleware:** middleware where clients ask for content by name or type, not by server address. It routes the request, caches popular content and returns it from the cache or the original provider.
- **RMI:** Java mechanism that lets an object in one JVM call methods of an object in another JVM as if it were local.
- **Remote interface:** interface that extends java.rmi.Remote, and each method declares RemoteException.
- **Stub / Skeleton:** the stub is the client-side proxy that marshals the call. The skeleton is the server-side part that unmarshals and invokes the real object (the skeleton is implicit in modern Java).
- **RMI Registry:** a naming service (default port 1099) where servers bind remote objects by name and clients look them up.
- **Marshaling / Unmarshaling:** converting parameters into a byte stream for transmission and converting them back.

- **Serialization:** converting an object into bytes (class implements Serializable). RMI uses it for parameters.
- **CORBA:** language-neutral standard for object brokering between heterogeneous systems.
- **ORB:** the core of CORBA. It finds the object, delivers requests and returns replies.
- **IDL:** language-neutral file that describes the interface. The idlj compiler generates Java stubs from it.
- **Naming Service (CORBA):** like the RMI registry, it maps names to object references. Started with orbd.
- **Clock synchronization:** keeping the clocks of different machines close to each other.
- **Berkeley algorithm:** the time server (master) polls all machines for their time, computes the average (ignoring outliers) and tells each machine how much to adjust (not the new time). It needs no UTC source.
- **Mutual exclusion:** only one process is in the critical section at a time.
- **Critical section:** code that accesses a shared resource.
- **Token ring algorithm:** processes are in a logical ring. A single token circulates, and only the holder can enter the critical section. If it does not need it, it passes the token on.
- **Election algorithm:** chooses one process as coordinator (leader). The highest process ID wins in Bully and Ring.
- **Bully algorithm:** a process that notices coordinator failure sends ELECTION to all higher IDs. If none reply, it becomes coordinator. If any reply OK, that one takes over. Message cost is high (O(n squared) worst case).
- **Ring election algorithm:** the ELECTION message with a list of live IDs travels round the ring. When it returns to the starter, the highest ID is chosen and a COORDINATOR message is circulated. Cost is about 2n messages.
- **Coordinator:** the leader process chosen by the election.
- **Replication:** keeping copies of data on several nodes for availability, fault tolerance and performance.
- **Distributed cache:** an in-memory store spread over several nodes that keeps frequently used data close to clients.
- **Cache hit / miss:** the data is found in the cache (hit) or is not (miss, so fetch from the origin).
- **Fault tolerance:** the system keeps working despite failures.
- **Checkpointing:** saving a consistent snapshot of the state periodically.

- **Rollback recovery:** after a failure, restore the last checkpoint and continue from there.
- **Recommendation system:** suggests items that best match user preferences (filter by genre, language, rating, then rank).
- **Metadata:** data about data (title, genre, language, rating).
- **Web cluster:** group of web servers acting as one site.
- **Load balancing:** spreading requests across servers. Round robin sends requests to servers in turn.
- **Proxy cache:** an intermediate server that stores responses and serves repeat requests, reducing response time and load.
- **Thread / multi-threaded server:** one thread per client request lets many clients be served concurrently. The synchronized keyword protects shared data.

B. Questions That Can Be Asked in the Practical

Assignment 1 (Java RMI, multi-threaded)

1. Write an RMI application where the server provides add, subtract and multiply, and many clients call it concurrently.
2. Write an RMI program to find the factorial or square of a number remotely.
3. Why does a remote method declare RemoteException? What is the use of the RMI registry? Which port does it use?
4. Show that the server handles multiple clients at the same time (print the thread name on the server).

Assignment 2 (CORBA)

1. Write an IDL file and a CORBA client and server for a Hello or Calculator service.
2. Which command generates stubs from IDL (idlj)? What does the ORB do? Which command starts the naming service (orbd)?
3. How does CORBA support heterogeneous systems (different languages and OSes)?

Assignment 3 (content-centric middleware)

1. Simulate a middleware that serves content from a cache or fetches it from the provider.
2. Add a cache limit with LRU eviction.
3. Print hit/miss count and the response source for each request.

Assignment 4 (Berkeley)

1. Implement the Berkeley algorithm for 4 processes with given clock values and show the adjustments.
2. Why is the average used? Why are adjustments sent and not the new time?
3. Redo it with a faulty clock that is excluded from the average.

Assignment 5 (token ring)

1. Simulate token ring mutual exclusion for N processes, where some request the critical section.
2. What happens if the token is lost? How many messages per entry?
3. Show starvation freedom (every process gets the token in turn).

Assignment 6 (Bully and Ring election)

1. Simulate the Bully algorithm when the coordinator crashes and a process starts an election.
2. Simulate the Ring algorithm for the same scenario and compare messages.
3. Which algorithm is better and why? What if two processes start the election together?

Assignment 7 (distributed cache)

1. Build 3 cache nodes with put and get, replicating each write to all nodes.
2. Simulate a node failure and show data is still served from another node.
3. Add consistent hashing or LRU (if time permits).

Assignment 8 (checkpointing)

1. Save the server state periodically to a file, simulate a crash and restore.
2. What is lost between the last checkpoint and the crash? How does a shorter interval help?
3. Difference between checkpointing and message logging.

Assignment 9 (video recommendation)

1. Store videos (title, genre, language, rating), accept preferences and print the matching videos sorted by rating.
2. Add a partial-match score if no exact match exists.

Assignment 10 (web cluster, LB, proxy cache)

1. Distribute requests over 3 servers using round robin and cache repeated URLs.
2. Show hit and miss counts. What if one server is down? (skip it in rotation)

General viva questions: RMI vs RPC, RMI vs CORBA, what is middleware, what is transparency, Network OS vs Distributed OS, why replicate, what is a logical clock, ring vs bully, what is a checkpoint, what is a proxy cache.

C. Programs to Memorise (with expected output)

Program 1: Multi-threaded Java RMI

Four files: Calc.java, CalcImpl.java, Server.java, Client.java. Compile all with `javac *.java`.

```
// Calc.java
import java.rmi.*;
public interface Calc extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

```
// CalcImpl.java
import java.rmi.*;
import java.rmi.server.*;
public class CalcImpl extends UnicastRemoteObject implements Calc {
    CalcImpl() throws RemoteException { super(); }
    public int add(int a, int b) {
        System.out.println(Thread.currentThread().getName() + " -> add(" + a
+ "," + b + ")");
        return a + b;
    }
}
```

```
// Server.java
import java.rmi.*;
import java.rmi.registry.*;
public class Server {
    public static void main(String[] args) throws Exception {
        LocateRegistry.createRegistry(1099);
        Naming.rebind("rmi://localhost:1099/calc", new CalcImpl());
        System.out.println("Server ready");
    }
}
```

```
// Client.java  (3 clients run concurrently as threads)
import java.rmi.*;
public class Client {
    public static void main(String[] args) throws Exception {
        for (int i = 1; i <= 3; i++) {
            final int n = i;
            new Thread(() -> {
                try {
                    Calc c = (Calc)
Naming.lookup("rmi://localhost:1099/calc");
                    System.out.println("Client" + n + ": Sum = " + c.add(n *
10, 5));
                } catch (Exception e) { e.printStackTrace(); }
            }).start();
        }
    }
}
```

Run: terminal 1: `java Server`, terminal 2: `java Client`.

Output

Server:

```
Server ready
RMI TCP Connection(2)-127.0.0.1 -> add(10,5)
RMI TCP Connection(3)-127.0.0.1 -> add(20,5)
RMI TCP Connection(4)-127.0.0.1 -> add(30,5)
```

Client (order may vary):

```
Client1: Sum = 15
Client2: Sum = 25
Client3: Sum = 35
```

Remember: remote interface extends Remote, every method throws RemoteException, impl extends UnicastRemoteObject, server does createRegistry and rebind, client does lookup. Different thread names on the server show that requests are handled concurrently.

Program 2: CORBA Hello (needs JDK 8 because CORBA was removed in JDK 11)

```
// Hello.idl
module HelloApp {
    interface Hello { string sayHello(); };
};
```

Generate stubs: `idlj -fall Hello.idl`

```
// HelloImpl.java
import HelloApp.*;
public class HelloImpl extends HelloPOA {
    public String sayHello() { return "Hello from CORBA server!"; }
}
```

```
// Server.java
import HelloApp.*;
import org.omg.CORBA.*;
import org.omg.CosNaming.*;
import org.omg.PortableServer.*;
public class Server {
    public static void main(String[] args) throws Exception {
        ORB orb = ORB.init(args, null);
        POA poa =
POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
        poa.the_POAManager().activate();
        Hello ref = HelloHelper.narrow(poa.servant_to_reference(new
HelloImpl()));
        NamingContextExt nc =
NamingContextExtHelper.narrow(orb.resolve_initial_references("NameService"));
        nc.rebind(nc.to_name("Hello"), ref);
        System.out.println("Server ready and waiting...");
        orb.run();
    }
}
```

```
// Client.java
import HelloApp.*;
import org.omg.CORBA.*;
import org.omg.CosNaming.*;
public class Client {
    public static void main(String[] args) throws Exception {
        ORB orb = ORB.init(args, null);
        NamingContextExt nc =
NamingContextExtHelper.narrow(orb.resolve_initial_references("NameService"));
        Hello h = HelloHelper.narrow(nc.resolve_str("Hello"));
        System.out.println(h.sayHello());
    }
}
```

Run order:

```
orbd -ORBInitialPort 1050 &
java Server -ORBInitialPort 1050 -ORBInitialHost localhost
java Client -ORBInitialPort 1050 -ORBInitialHost localhost
```

Output

```
Server ready and waiting...      (server)
Hello from CORBA server!         (client)
```

Remember: IDL, then idlj, then servant extends the POA class, server activates the POA, registers with the Naming Service and calls orb.run(). The client resolves the name and narrows to the interface.

Program 3: Content-centric middleware (cache or provider)

```
import java.util.*;
public class ContentMiddleware {
    static Map<String, String> provider = new HashMap<>();
    static Map<String, String> cache = new HashMap<>();
    static int hits = 0, misses = 0;

    static String request(String content) {
        if (cache.containsKey(content)) {
            hits++;
            System.out.println("CACHE HIT  -> " + content);
            return cache.get(content);
```

```
        }
        misses++;
        System.out.println("CACHE MISS -> fetching '" + content + "' from
provider");
        if (!provider.containsKey(content)) return "404 Content Not Found";
        cache.put(content, provider.get(content));
        return provider.get(content);
    }

    public static void main(String[] args) {
        provider.put("video1", "Video One Data");
        provider.put("news", "Today's News");
        String[] reqs = {"video1", "video1", "news", "xyz", "news"};
        for (String r : reqs)
            System.out.println("Response: " + request(r));
        System.out.println("Hits=" + hits + " Misses=" + misses);
    }
}
```

Output

```
CACHE MISS -> fetching 'video1' from provider
Response: Video One Data
CACHE HIT  -> video1
Response: Video One Data
CACHE MISS -> fetching 'news' from provider
Response: Today's News
CACHE MISS -> fetching 'xyz' from provider
Response: 404 Content Not Found
CACHE HIT  -> news
Response: Today's News
Hits=2 Misses=3
```

Remember: a request goes to the middleware, which checks the cache first, and on a miss asks the provider and stores the result. Routing is done by content name.

(Programs 4 to 10 continue below.)

Program 4: Berkeley clock synchronization

```
public class Berkeley {
    public static void main(String[] args) {
        int[] t = {100, 110, 95, 115};   // P0 is the time server (master)
        int master = t[0], sum = 0;
        for (int i = 0; i < t.length; i++) {
            int diff = t[i] - master;
            System.out.println("P" + i + " time=" + t[i] + " diff=" + diff);
            sum += diff;
        }
        int avg = sum / t.length;
        System.out.println("Average diff = " + avg);
        for (int i = 0; i < t.length; i++) {
            int adjust = avg - (t[i] - master);
            t[i] += adjust;
            System.out.println("P" + i + " adjust=" + adjust + " new time=" +
t[i]);
        }
    }
}
```

Output

```
P0 time=100 diff=0
P1 time=110 diff=10
P2 time=95 diff=-5
P3 time=115 diff=15
Average diff = 5
P0 adjust=5 new time=105
P1 adjust=-5 new time=105
P2 adjust=10 new time=105
P3 adjust=-10 new time=105
```

Remember: the master computes each difference from its own clock, averages them (master included), then sends each machine the adjustment (average minus its own difference). All clocks end up equal.

Program 5: Token ring mutual exclusion

```
public class TokenRing {
    public static void main(String[] args) throws Exception {
        int n = 4;
        boolean[] wants = {false, true, false, true};   // who needs the
critical section
        int token = 0;
        for (int i = 0; i < n; i++) {
            int p = token;
            if (wants[p]) {
                System.out.println("P" + p + " has token -> ENTERS critical
section");
                Thread.sleep(300);
                System.out.println("P" + p + " exits critical section");
                wants[p] = false;
            } else {
                System.out.println("P" + p + " has token -> no request,
passes it on");
            }
            token = (p + 1) % n;
            System.out.println("Token passed to P" + token);
        }
    }
}
```

Output

```
P0 has token -> no request, passes it on
Token passed to P1
P1 has token -> ENTERS critical section
P1 exits critical section
Token passed to P2
P2 has token -> no request, passes it on
Token passed to P3
P3 has token -> ENTERS critical section
P3 exits critical section
Token passed to P0
```

Remember: one token only, so only one process is in the critical section at a time. Token passes to (i+1) mod n. No starvation, but a lost token must be regenerated.

Program 6: Bully and Ring election (P5 coordinator has crashed, P2 starts)

```
import java.util.*;
public class Election {
    static int N = 5;
    static boolean[] alive = {false, true, true, true, true, false}; // index
1..5, P5 is down

    static void bully(int s) {
        boolean got = false;
        for (int i = s + 1; i <= N; i++) {
            System.out.println("P" + s + " sends ELECTION to P" + i);
            if (alive[i]) { System.out.println("P" + i + " replies OK"); got
= true; }
            else System.out.println("P" + i + " no response (down)");
        }
        if (!got) {
            System.out.println("P" + s + " is new COORDINATOR");
            for (int i = 1; i <= N; i++)
                if (i != s && alive[i]) System.out.println("P" + s + " sends
COORDINATOR to P" + i);
            return;
        }
        for (int i = s + 1; i <= N; i++) if (alive[i]) { bully(i); return; }
    }

    static int next(int i) {
        int n = i % N + 1;
        while (!alive[n]) n = n % N + 1;
        return n;
    }

    static void ring(int s) {
        List<Integer> list = new ArrayList<>();
        list.add(s);
        int nxt = next(s);
        while (nxt != s) {
            System.out.println("ELECTION" + list + " -> P" + nxt);
            list.add(nxt);
            nxt = next(nxt);
        }
        int coord = Collections.max(list);
        System.out.println("Back at P" + s + ": " + list + ", COORDINATOR =
```

```
P" + coord);
    }

    public static void main(String[] a) {
        System.out.println("--- BULLY ---");
        bully(2);
        System.out.println("--- RING ---");
        ring(2);
    }
}
```

Output

```
--- BULLY ---
P2 sends ELECTION to P3
P3 replies OK
P2 sends ELECTION to P4
P4 replies OK
P2 sends ELECTION to P5
P5 no response (down)
P3 sends ELECTION to P4
P4 replies OK
P3 sends ELECTION to P5
P5 no response (down)
P4 sends ELECTION to P5
P5 no response (down)
P4 is new COORDINATOR
P4 sends COORDINATOR to P1
P4 sends COORDINATOR to P2
P4 sends COORDINATOR to P3
--- RING ---
ELECTION[2] -> P3
ELECTION[2, 3] -> P4
ELECTION[2, 3, 4] -> P1
Back at P2: [2, 3, 4, 1], COORDINATOR = P4
```

Remember: Bully sends to higher IDs only, so more messages. Ring passes one message round the ring collecting live IDs and picks the maximum, so fewer messages. Both elect the highest live ID (P4 here).

Program 7: Distributed cache with replication

```
import java.util.*;
class Node {
    String name;
    Map<String, String> cache = new HashMap<>();
    boolean up = true;
    Node(String n) { name = n; }
}
public class DistCache {
    static List<Node> nodes = new ArrayList<>();

    static void put(String k, String v) {          // replicate to all live
nodes
        for (Node n : nodes)
            if (n.up) { n.cache.put(k, v); System.out.println("Stored " + k +
" in " + n.name); }
    }
    static String get(String k) {
        for (Node n : nodes)
            if (n.up && n.cache.containsKey(k)) {
                System.out.println("HIT on " + n.name);
                return n.cache.get(k);
            }
        System.out.println("MISS");
        return null;
    }
    public static void main(String[] a) {
        nodes.add(new Node("Node1"));
        nodes.add(new Node("Node2"));
        nodes.add(new Node("Node3"));
        put("user1", "Alice");
        System.out.println(get("user1"));
        nodes.get(0).up = false;
        System.out.println("Node1 FAILED");
        System.out.println(get("user1"));
        System.out.println(get("user9"));
    }
}
```

Output

```
Stored user1 in Node1
Stored user1 in Node2
Stored user1 in Node3
HIT on Node1
Alice
Node1 FAILED
HIT on Node2
Alice
MISS
null
```

Remember: write goes to every node (replication), read goes to the first live node that has the key, so a failed node does not lose data.

Program 8: Checkpoint and recovery

```
import java.io.*;
public class Checkpoint {
    static int counter = 0;

    static void save() throws Exception {
        try (ObjectOutputStream o = new ObjectOutputStream(new
FileOutputStream("cp.dat"))) {
            o.writeInt(counter);
        }
        System.out.println("Checkpoint saved: " + counter);
    }
    static void restore() throws Exception {
        try (ObjectInputStream i = new ObjectInputStream(new
FileInputStream("cp.dat"))) {
            counter = i.readInt();
        }
        System.out.println("Restored from checkpoint: " + counter);
    }
    public static void main(String[] args) throws Exception {
        boolean crashed = false;
        while (counter < 10) {
            counter++;
            System.out.println("Processing " + counter);
            if (counter % 3 == 0) save();                // checkpoint every
3 steps
            if (counter == 8 && !crashed) {              // simulated
```

```
failure
                crashed = true;
                System.out.println("*** SERVER CRASHED ***");
                counter = 0;                              // state lost
                restore();
            }
        }
        System.out.println("Done, final state = " + counter);
    }
}
```

Output

```
Processing 1
Processing 2
Processing 3
Checkpoint saved: 3
Processing 4
Processing 5
Processing 6
Checkpoint saved: 6
Processing 7
Processing 8
*** SERVER CRASHED ***
Restored from checkpoint: 6
Processing 7
Processing 8
Processing 9
Checkpoint saved: 9
Processing 10
Done, final state = 10
```

Remember: save state to a file at intervals, on failure reload the last file. Only work after the last checkpoint (steps 7 and 8) is redone. If the examiner wants a real client-server version, wrap this in a ServerSocket loop: the client sends a number, the server adds it to counter, and the same save() and restore() are used.

Program 9: Video recommendation by user preference

```
import java.util.*;
class Video {
    String title, genre, lang;
    double rating;
    Video(String t, String g, String l, double r) { title = t; genre = g;
lang = l; rating = r; }
    public String toString() { return title + " | " + genre + " | " + lang +
" | " + rating; }
}
public class VideoRec {
    public static void main(String[] args) {
        List<Video> v = new ArrayList<>();
        v.add(new Video("Inception", "SciFi", "English", 8.8));
        v.add(new Video("3 Idiots", "Comedy", "Hindi", 8.4));
        v.add(new Video("Interstellar", "SciFi", "English", 8.6));
        v.add(new Video("Sairat", "Romance", "Marathi", 8.0));

        Scanner s = new Scanner(System.in);
        System.out.print("Genre: ");       String g = s.nextLine();
        System.out.print("Language: ");    String l = s.nextLine();
        System.out.print("Min rating: ");  double r = s.nextDouble();

        List<Video> res = new ArrayList<>();
        for (Video x : v)
            if (x.genre.equalsIgnoreCase(g) && x.lang.equalsIgnoreCase(l) &&
x.rating >= r)
                res.add(x);
        res.sort((p, q) -> Double.compare(q.rating, p.rating));   // best
rating first

        System.out.println("Recommended videos:");
        if (res.isEmpty()) System.out.println("No match found");
        for (Video x : res) System.out.println(x);
    }
}
```

Sample run

```
Genre: SciFi
Language: English
Min rating: 8.5
Recommended videos:
Inception | SciFi | English | 8.8
Interstellar | SciFi | English | 8.6
```

Remember: Video class (metadata), list of videos, read preferences, filter, sort by rating, print.

Program 10: Web cluster with load balancer and proxy cache

```
import java.util.*;
public class WebCluster {
    static String[] servers = {"Server1", "Server2", "Server3"};
    static int rr = 0;
    static Map<String, String> proxyCache = new HashMap<>();
    static int hits = 0, misses = 0;

    static String handle(String url) {
        if (proxyCache.containsKey(url)) {
            hits++;
            System.out.println("[Proxy] CACHE HIT  " + url);
            return proxyCache.get(url);
        }
        misses++;
        String s = servers[rr++ % servers.length];          // round robin
        System.out.println("[LB] " + url + " -> " + s);
        String resp = "Page '" + url + "' served by " + s;
        proxyCache.put(url, resp);
        return resp;
    }
    public static void main(String[] args) {
        String[] reqs = {"/home", "/about", "/home", "/contact", "/about",
"/news"};
        for (String r : reqs) System.out.println(handle(r));
        System.out.println("Hits=" + hits + " Misses=" + misses);
    }
}
```

Output

```
[LB] /home -> Server1
Page '/home' served by Server1
[LB] /about -> Server2
Page '/about' served by Server2
[Proxy] CACHE HIT  /home
Page '/home' served by Server1
[LB] /contact -> Server3
Page '/contact' served by Server3
[Proxy] CACHE HIT  /about
Page '/about' served by Server2
[LB] /news -> Server1
Page '/news' served by Server1
Hits=2 Misses=4
```

Remember: proxy cache is checked first, and on a miss the load balancer picks the next server by round robin (index mod number of servers).

D. Last-Minute Tips

- Programs 3, 4, 5, 6, 7, 8, 9 and 10 are single-file simulations. If the examiner insists on real networking, use `ServerSocket` on the server and `Socket` on the client with `DataInputStream` and `DataOutputStream`, then put the same logic inside.
- For RMI, always start the server first. If you get a connection error, check that the registry port 1099 is free and that both sides use the same name string.
- For CORBA, use JDK 8 (or add the JacORB library). Start orbd before the server.
- Practise writing programs 1, 4, 5 and 6 from memory first. They are the most commonly asked.
- Know one line of explanation per program: what problem it solves, the algorithm steps, and the output meaning.